

Why do we need Selenium Waits?



Contents

- Web Elements
- Types of Web Elements Actions
- `time.sleep()` and its disadvantages
- Implicit Wait
- Explicit Wait
- Comparison between Implicit and Explicit Wait
- Fluent Wait

Why do we need selenium waits?

Introduction

- Majority of modern application's front-end is built on JavaScript or Ajax, using frameworks such as React, Angular, or any other which takes a certain time for the web elements to load on the page, whenever that page is loaded or refreshed.
- Hence, in case you tend to locate an element in your script which is yet to load on the page, selenium will throw you 'Exception' message.
- Three different exceptions.
 1. The Element we try to interact/find is not at all present in the DOM.
 2. Element is present but not visible.
 3. Element is present but not interactable.

Why do we need selenium waits?

An Example Scenario

- Below code snippet will help you showcase the same problem as you execute automation testing with Selenium.

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://omayo.blogspot.com/")

driver.find_element(By.CLASS_NAME, value="dropbtn").click()

driver.find_element(By.LINK_TEXT, value="Facebook").click()
```

Why do we need selenium waits?

An Example Scenario

Output:

```
[Running] python -u "c:\Users\Arun Kumar\VSCodePython\waits.py"
Traceback (most recent call last):
  File "c:\Users\Arun Kumar\VSCodePython\waits.py", line 10, in <module>
    driver.find_element(By.LINK_TEXT, value="Facebook").click()
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Arun Kumar\AppData\Local\Programs\Python\Python312\Lib\site-
packages\selenium\webdriver\remote\webdriver.py", line 742, in find_element
    return self.execute(Command.FIND_ELEMENT, {"using": by, "value": value})["value"]
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Arun Kumar\AppData\Local\Programs\Python\Python312\Lib\site-
packages\selenium\webdriver\remote\webdriver.py", line 348, in execute
    self.error_handler.check_response(response)
  File "C:\Users\Arun Kumar\AppData\Local\Programs\Python\Python312\Lib\site-
packages\selenium\webdriver\remote\errorhandler.py", line 229, in check_response
    raise exception_class(message, screen, stacktrace)
selenium.common.exceptions.NoSuchElementException: Message: no such element: Unable to locate
element: {"method":"link text","selector":"Facebook"}
(Session info: chrome=121.0.6167.85); For documentation on this error, please visit:
https://www.selenium.dev/documentation/webdriver/troubleshooting/errors#no-such-element-exception
```

Why do we need selenium waits?

An Example Scenario

Output:

← → ↻ 🔒 omayo.blogspot.com

Chrome is being controlled by automated test software.

TextWillBeDisplayedWithDelay
This text is displayed after 10 seconds of wait.

Popup Window
[Open a popup window](#)

UploadFile

Choose File No file chosen

TimerEnableButton

Button3

Disable Enable Button

My Button

Click the button try it button to disable the button after 3 seconds.

Try it

Button X Button Y

LoginSection
Login page
Simple Login Page
Username Password

Login Cancel

www.selenium-by-arun.blogspot.com

Select a vehicle
Bike: ☐ Bicycle: ☒ Car: ☐

Select multiple options
Pen: ☐ Book: ☒ Laptop: ☐ Bag: ☐

TestDoubleClick
Double-click

Delayed-Button-Dropdown

Dropdown

Facebook
Flipkart
Gmail

6 Handling Web Elements and Mouse Events | © Expleo | Internal | Version 1.0

[expleo]

Why do we need selenium waits?

Introduction

- Now, since the page hasn't loaded yet, the script failed to find the 'Facebook' button. Resulting in throwing a 'NoSuchElementException'.
- Selenium wait for a page to load helps in resolving this issue.
- There are different types of Selenium waits like Implicit wait and Explicit wait, that ensures the elements are loaded into the page before they are discovered by the Selenium script for further actions.

How to avoid browser notifications?



How to avoid browser notification

To avoid notification created by browser

```
options = webdriver.ChromeOptions()
```

```
options.add_argument('--disable-notifications')
```

```
driver = webdriver.Chrome(options=options)
```

Types of Selenium Waits



Types of Selenium Waits

Selenium Waits

- When performing automation testing with Selenium, we use the following types of waits as we generate our Selenium script:
- Synchronize between script execution and application, we need to wait after performing appropriate actions.
- `time.sleep()` method (Hard Wait)
- Implicit Wait
- Explicit Wait
- Fluent Wait

Time.sleep() and its disadvantages



Time.sleep

Time.sleep()

- sleep() is a static method that belongs to the time class.
- This method can be called using the reference of the class name i.e Time.
- If you use Time.sleep while performing automation testing with Selenium, then this method will stop the execution of the script for the specified duration of time, irrespective of whether the element is found or not on the web page.
- It accepts the time duration in milliseconds.
- The syntax for the same is:

Time.sleep(3);

An Example Scenario

- Below code snippet will help you showcase the same problem as you execute automation testing with Selenium.

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()

driver.get("https://omayo.blogspot.com/")

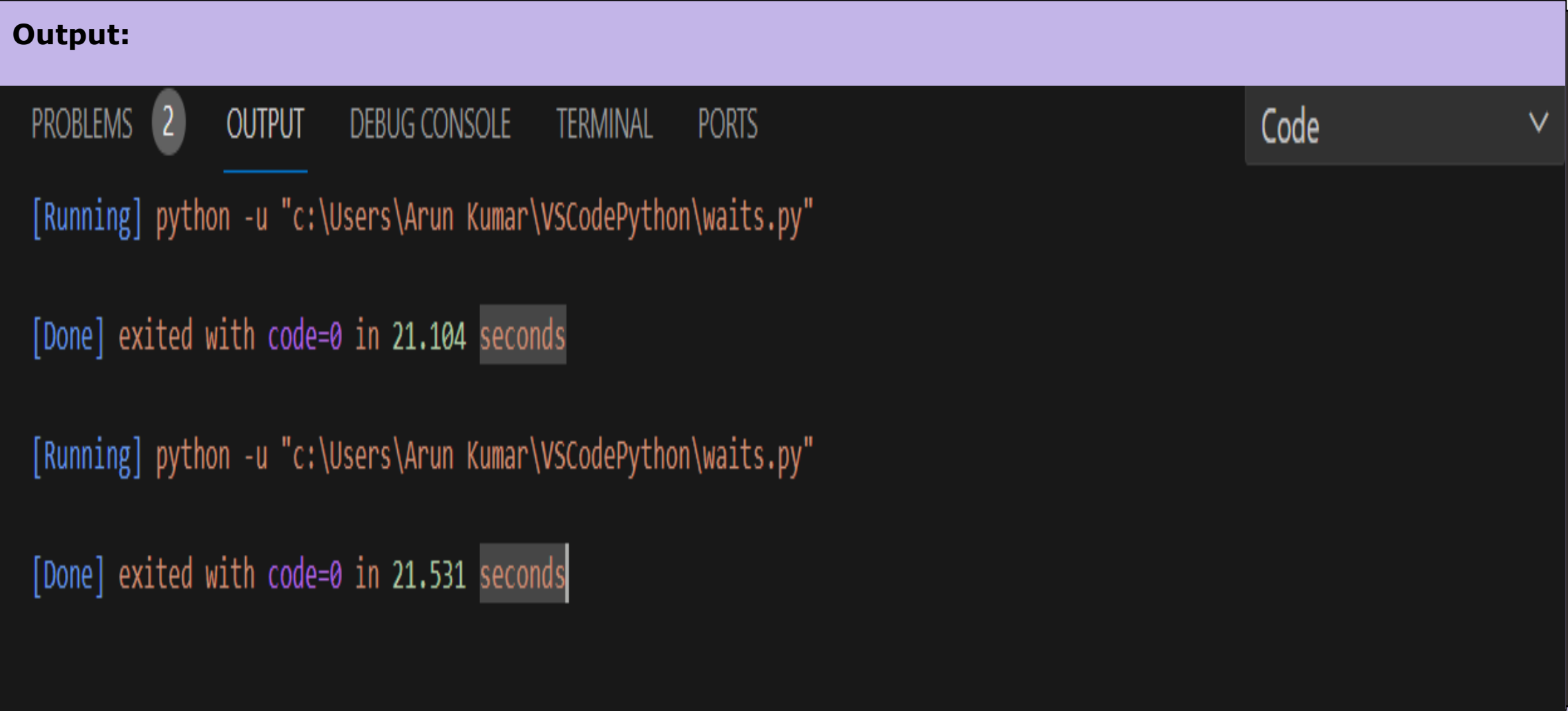
driver.find_element(By.CLASS_NAME, value="dropbtn").click()

#Hard Wait
time.sleep(5)

driver.find_element(By.LINK_TEXT, value="Facebook").click()
```

An Example Scenario

Output:



The screenshot shows the VS Code interface with the 'OUTPUT' tab selected. The output window displays two consecutive runs of a Python script. Each run starts with a '[Running]' status followed by the command 'python -u "c:\Users\Arun Kumar\VSCodePython\waits.py"'. After a short delay, the status changes to '[Done]' followed by 'exited with code=0 in 21.104 seconds' and '21.531 seconds' respectively. The text is color-coded: '[Running]' and '[Done]' are blue, 'python' is orange, and the file path and time are green.

```
[Running] python -u "c:\Users\Arun Kumar\VSCodePython\waits.py"

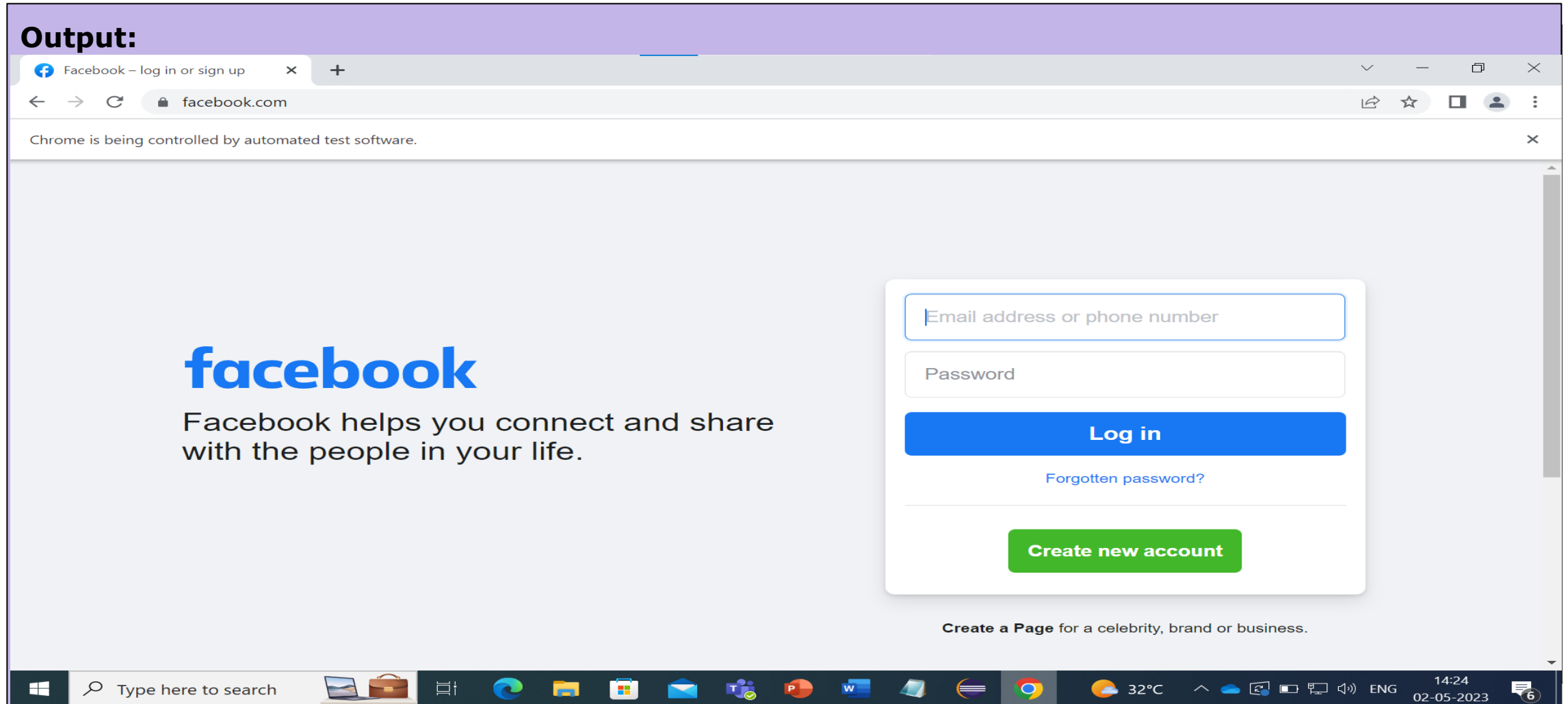
[Done] exited with code=0 in 21.104 seconds

[Running] python -u "c:\Users\Arun Kumar\VSCodePython\waits.py"

[Done] exited with code=0 in 21.531 seconds
```

Why do we need selenium waits?

An Example Scenario



Why Using Time.sleep() Is Not A Good Idea?

- Selenium Webdriver waits for the specified time, irrespective of the element is found or not.
- In case the element is found much before the specified duration, the script will still wait for the time duration to elapse, thereby increasing the execution time of the script.
- If the element to be present does not appear after a static time and keep changing, then you will never know an estimated time needed for the sleep function.
- Time.sleep is intended only for the element it is written prior to. In case you have two to four elements which need to wait for a certain duration to load, you need to specify Time.sleep as many times in that case.

Implicit Wait



Implicit Wait

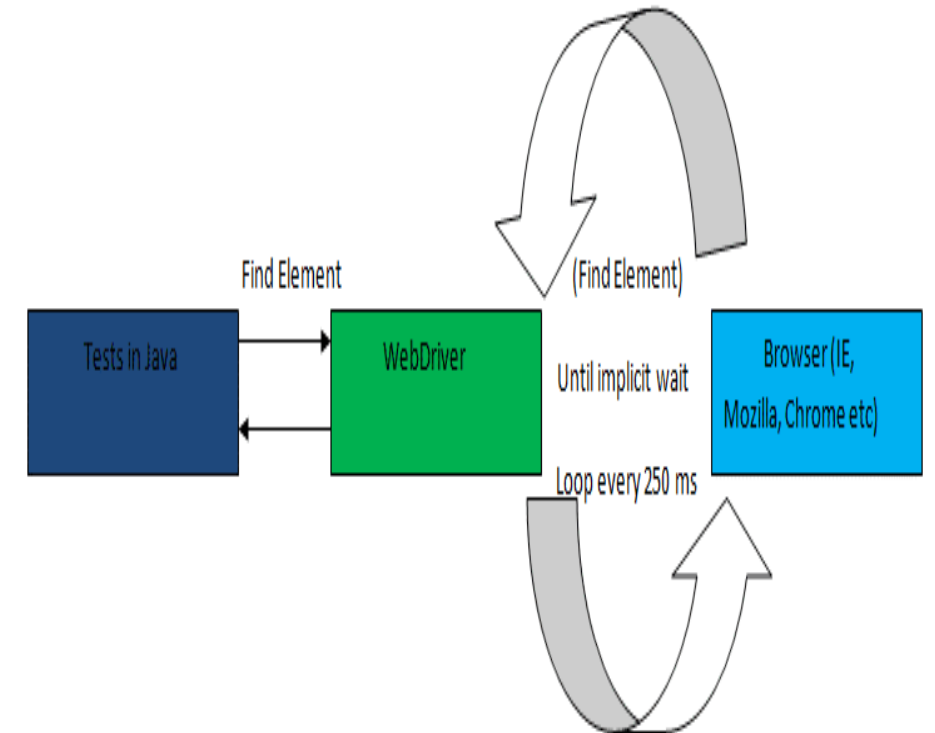
Implicit Wait

- Selenium has overcome the problems provided by `Time.sleep()` and have come up with two Selenium waits for page load.
- One of which is **Implicit wait** which allows you to halt the `WebDriver` for a particular period of time until the `WebDriver` locates a desired element on the web page.
- The key point to note here is, unlike `Time.sleep()`, it does not wait for the complete duration of time.
- This is why Implicit wait is also referred to as dynamic wait. If it does not find the element in the specified duration, it throws `ElementNotVisibleException`.

Implicit Wait

Implicit Wait

- Another interesting thing to note about Implicit wait is that it is applied globally, which makes it a better option than `Time.sleep()`.
- Meaning you only need to write it one time and it gets applicable for all of the web elements specified on a script throughout the WebDriver instance.



Implicit Wait

Implicit Wait

- Syntax:

```
driver.implicitly_wait(Time Interval in Seconds)
```

- Example:

```
driver.implicitly_wait(5)
```

Implicit Wait

An Example Scenario

- Below code snippet will help you showcase the same problem as you execute automation testing with Selenium.

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.maximize_window()

#Implicit Wait - Global Wait
driver.implicitly_wait(5)

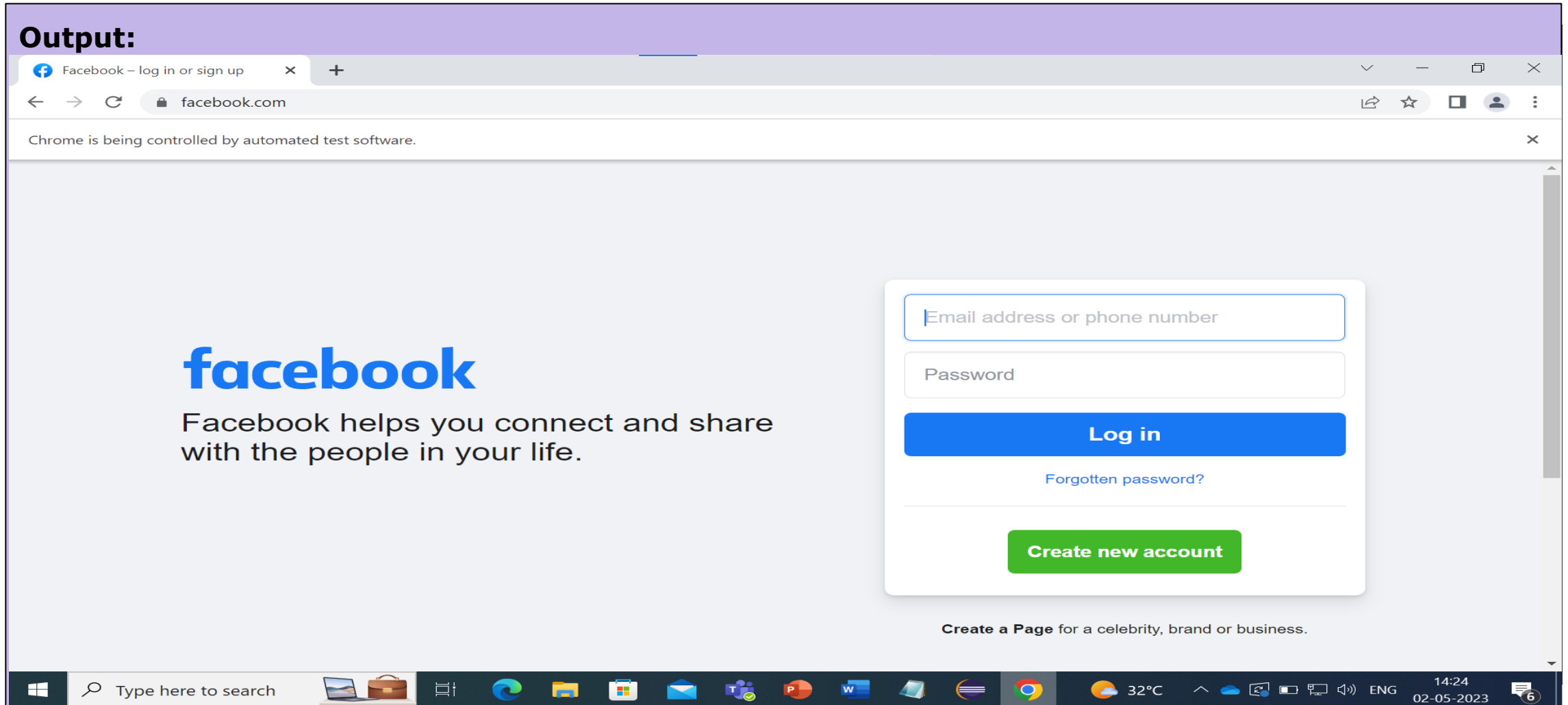
driver.get("https://omayo.blogspot.com/")

driver.find_element(By.CLASS_NAME, value="dropbtn").click()

driver.find_element(By.LINK_TEXT, value="Facebook").click()
```

Implicit Wait

Example



Explicit Wait



Explicit Wait

Explicit Wait

- Now, here we are aware of the fact, that pages shall we loaded in a certain duration, but what if we do not know the element to be visible/clickable at loading time.
- As in the time of its appearance is dynamic and keeps on changing from time to time. In this case, Explicit wait will help you overcome this problem.
- The Explicit wait is another one of the dynamic Selenium waits.
- Explicit wait help to stop the execution of the script based on a certain condition for a specified amount of time.
- Once the time goes overboard, you will get the `ElementNotVisibleException`.
- In a scenario where you do not know the amount of time to wait for, this explicit wait comes in handy.

Explicit Wait

Explicit Wait

- Using conditions like `visibility_of_element_located` or `element_to_be_clickable`, one can wait for the specified duration.
- One can use these predefined methods using the combination of classes `WebDriverWait` and `expected_conditions`.
- In order to use this case, import the below packages in your class:

```
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
```

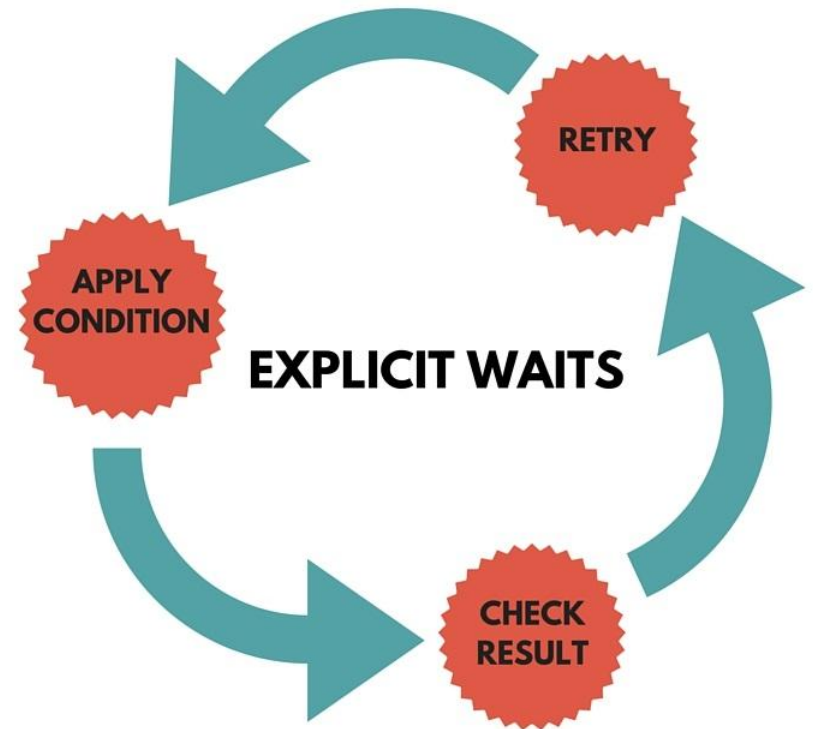
Explicit Wait

Explicit Wait

- Then, Initialize A Wait Object using WebDriverWait Class.

```
wait = WebDriverWait(driver,10)
```

- Here, the reference variable is named <wait> for the <WebDriverWait> class.
- It is instantiated using the WebDriver instance.
- The maximum wait time must be set for the execution to layoff.
- Note that the wait time is measured in seconds.



Explicit Wait

- In order to use the predefined methods of the **expected_condition Class**, we will use the wait reference variable as below:

Types of Expected Conditions:

- Below are the few types of expected conditions commonly used as you perform automation testing with Selenium.
- **visibility_of_element_located** - Verifies if the given element is present or not
- **alert_is_present** - Verifies if the alert is present or not.
- **element_to_be_clickable** - Verifies if the given element is present/clickable on the screen
- **text_to_be_present_in_element** - Verifies the given element have the required text or not
- **title_contains** - Verify the condition wait for a page that has a given title

Explicit Wait

Explicit Wait

Syntax:

- `wait.until(expected_conditions.visibility_of_element_located((Reference of Element to be located using locator)))`

Example:

```
wait = WebDriverWait(driver,10)
```

```
facebook_option = wait.until(expected_conditions.visibility_of_element_located((By.LINK_TEXT,  
"Facebook")))
```

```
facebook_option.click()
```

Explicit Wait

An Example Scenario

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait

driver = webdriver.Chrome()
driver.maximize_window()

driver.get("https://omayo.blogspot.com/")

driver.find_element(By.CLASS_NAME, value="dropbtn").click()

wait = WebDriverWait(driver,10)

facebook_option = wait.until(expected_conditions.visibility_of_element_located((By.LINK_TEXT,
"Facebook"))))

facebook_option.click()
```

Comparison between Implicit and Explicit Wait



Implicit Wait versus Explicit Wait

IMPLICIT WAIT	EXPLICIT WAIT
It is by default applied to all the elements in the script.	It is applicable to only a certain element which is specific to a certain condition.
We cannot wait based on a specified condition like element selectable/clickable unlike explicit.	In explicit, we can specify the wait based on a specific condition.
It is usually used when you are sure the element may be visible in a certain time	It is usually used, when you are not aware of the time of the element visibility. It is subjected to dynamic nature.

Fluent Wait



Fluent Wait

- The Fluent wait is similar to explicit wait in terms of its functioning.
- In Fluent wait, you perform a Selenium wait for an element when you are not aware of the time it may take to be visible or clickable.
- The few differential factors that Fluent wait offers are:
 - **The polling frequency:**
 - In case of Explicit wait, this polling frequency is by default 500 milliseconds.
 - Using Fluent wait, you can change this polling frequency based on your needs, i.e you can tell your script to keep checking on an element after every 'x' seconds.

Fluent Wait

- **Ignore Exception :**
 - During polling, in case you do not find an element, you can ignore any exception like 'NoSuchElement' exception etc.
- Fluent Wait commands are most useful when interacting with web elements that can take longer durations to load. This is something that often occurs in Ajax applications.
- Fluent waits are also sometimes called smart waits because they don't wait out the entire duration defined in the code. Instead, the test continues to execute as soon as the element is detected – as soon as the condition specified in `.until(YourCondition)` method becomes true.

Fluent Wait

An Example Scenario

```
import time
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions
from selenium.webdriver.support.wait import WebDriverWait
from selenium.common.exceptions import NoSuchElementException

driver = webdriver.Chrome()
driver.maximize_window()

driver.get("https://omayo.blogspot.com/")

driver.find_element(By.CLASS_NAME, value="dropbtn").click()

#Fluent Wait

wait = WebDriverWait(driver,timeout=30,poll_frequency=5,ignored_exceptions=[NoSuchElementException])

facebook_option = wait.until(expected_conditions.visibility_of_element_located((By.LINK_TEXT, "Facebook")))

facebook_option.click()
```